

```
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
```

```
#define MAXPAROLA 30
#define MAXRIGA 80
```

```
int main(int argc, char *argv[])
{
    int freq[MAXPAROLA]; /* vettore di contatori
delle frequenze delle lunghezze delle parole */
    char riga[MAXRIGA];
    int i, inizio, lunghezza;
    FILE *f;
```

```
for(i=0; i<MAXPAROLA; i++)
    freq[i]=0;
```

```
if(argc != 2)
```

```
{
    printf(stderr, "ERRORE, serve un parametro con il nome del file\n");
    exit(1);
}
```

```
f = fopen(argv[1], "r");
if(f==NULL)
```

```
{
    printf(stderr, "ERRORE, impossibile aprire il file %s\n", argv[1]);
    exit(1);
}
```

```
while( fgets( riga, MAXRIGA, f ) != NULL )
```



Synchronization

Synchronization in C

Stefano Quer

Dipartimento di Automatica e Informatica
Politecnico di Torino

License Information

This work is licensed under the license



Attribution-NonCommercial-NoDerivatives 4.0 International

This license requires that reusers give credit to the creator. It allows reusers to copy and distribute the material in any medium or format in unadapted form and for noncommercial purposes only.

① **BY:** Credit must be given to you, the creator.

② **NC:** Only noncommercial use of your work is permitted.

Noncommercial means not primarily intended for or directed towards commercial advantage or monetary compensation.

③ **ND:** No derivatives or adaptations of your work are permitted.

To view a copy of the license, visit:

<https://creativecommons.org/licenses/by-nc-nd/4.0/?ref=chooser-v1>

Semaphore implementations

- ❖ Mutexes in C are
 - Represented by object of type **mtx_t**
 - Defined in `threads.h`, i.e., insert
 - `#include <threads.h>`
- ❖ See documentation for
 - Atomic operation and fences (barriers) in C

Barriers are introduced in
unit 06 section 07

Mutual exclusion

For more operations see the reference documentation

Type	Meaning
<code>int mtx_init(mtx_t *mtx, int muxtype);</code>	Create a mutex (mtx) with some properties (muxtype).
<code>void mtx_destroy(mtx *mtx);</code>	Destroy the mutex pointed by mtx.
<code>int mtx_lock(mtx_t *mtx);</code>	Blocks the calling thread until it obtain the mutex referenced by mtx.
<code>int mtx_trylock(mtx_t *mtx);</code>	Try to obtain the mutex referenced by mtx but it does not block the thread.
<code>int mtx_timedlock(mtx_t *mtx, cont struct timespec *ts);</code>	Try to obtain the mutex referenced by mtx but it blocks the thread only for a specific time.
<code>int mtx_unlock(mtx_x *mtx);</code>	Releases the mutex referred by mtx.